Task 1 — Enhanced Human Body Landmark Detection System

Name: Praneetha AS (aspraneetha29@gmail.com)

Role: AI/ML Intern

Date: 15/05/2026

Project Folder: Raritone

1. Objective

The objective of this task was to improve the performance and accuracy of Human Body Landmark Detection using Artificial Intelligence and Computer Vision techniques.

The task focused on:

- improving pose tracking accuracy
- reducing landmark flickering
- improving skeleton visualization
- increasing landmark stability
- optimizing real-time body tracking

The project was implemented using:

- Python
- OpenCV
- MediaPipe

---

2. Introduction to Human Body Landmark Detection

Human Body Landmark Detection is a computer vision process used to identify and track important body points from images and videos.

AI systems detect landmarks such as:

- shoulders
- elbows
- wrists
- hips
- knees
- ankles

The detected landmarks are connected together to create a digital skeleton representation of the human body.

Body landmark detection is widely used in:

- AI fitness applications
- sports analysis
- healthcare monitoring
- virtual fitting systems
- gesture recognition
- motion tracking systems

Figure 1 — Human Body Landmark Detection

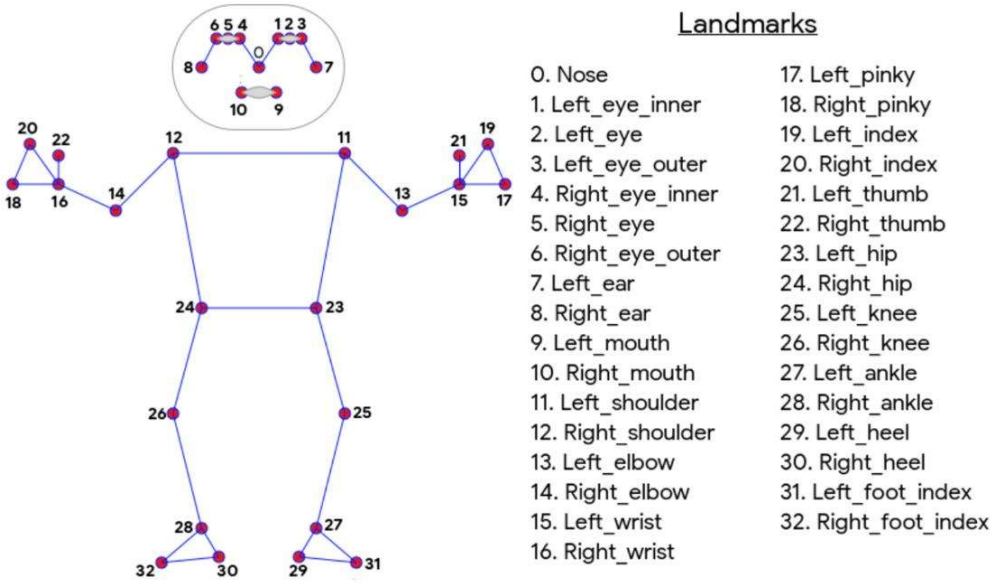


Figure 1. Landmarks

---

3. Improvements Implemented

Compared to the previous implementation, multiple improvements were added to enhance the quality and stability of pose detection.

Day 2 System	Improved Day 5 System
Basic landmark detection	Improved landmark accuracy
Default tracking settings	Increased detection confidence
Basic skeletal visualization	Smooth landmark tracking
Standard pose estimation	Higher model complexity
Simple webcam tracking	Optimized real-time processing
Minor landmark flickering	Reduced landmark instability

---

4. Technologies Used

Technology	Purpose
Python	Programming Language
OpenCV	Real-time image processing
MediaPipe	Human pose estimation
Artificial Intelligence	Landmark analysis
Computer Vision	Body tracking

---

5. Improved Pose Detection Workflow

The following workflow was used to improve landmark detection performance.

Step 1 — Webcam Input Capture

OpenCV captured real-time webcam frames.

Step 2 — Frame Optimization

Frames were resized for smoother AI processing.

Step 3 — RGB Conversion

MediaPipe processed RGB images for pose estimation.

Step 4 — Landmark Detection

The AI model identified body landmarks dynamically.

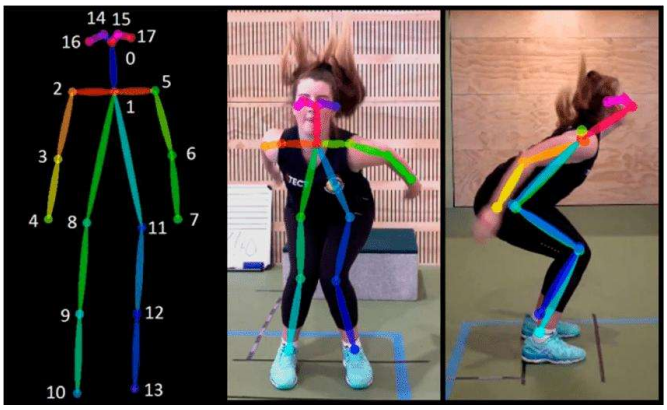
Step 5 — Skeleton Visualization

Detected landmarks were connected to create skeletal tracking.

Step 6 — Real-Time Movement Tracking

The system continuously tracked body movement and updated landmark positions dynamically.

Figure 2 — Real-Time Skeleton Tracking



---

6. Code Implementation

The following Python program was developed to improve real-time body landmark detection using MediaPipe and OpenCV.

Improved Body Landmark Detection Code

```
import cv2
import mediapipe as mp

# Initialize MediaPipe Pose
mp_pose = mp.solutions.pose
mp_draw = mp.solutions.drawing_utils

pose = mp_pose.Pose(
   static_image_mode=False,
   model_complexity=2,
   smooth_landmarks=True,
   min_detection_confidence=0.7,
   min_tracking_confidence=0.7
)

# Start Webcam
cap = cv2.VideoCapture(0)

while True:
   success, frame = cap.read()
```

```
    if not success:
        break

    # Resize Frame
    frame = cv2.resize(frame, (900, 700))

    # Convert BGR to RGB
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Process Pose Detection
    results = pose.process(rgb)

    # Draw Landmarks
    if results.pose_landmarks:
        mp_draw.draw_landmarks(
            frame,
            results.pose_landmarks,
            mp_pose.POSE_CONNECTIONS,
            mp_draw.DrawingSpec(color=(0,255,0), thickness=2, circle_radius=2),
            mp_draw.DrawingSpec(color=(0,0,255), thickness=2)
        )

    # Display Output
    cv2.imshow("Improved Body Landmark Detection", frame)

    # Exit Key
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

---

7. Improvements Added in the Code

The following enhancements were added:

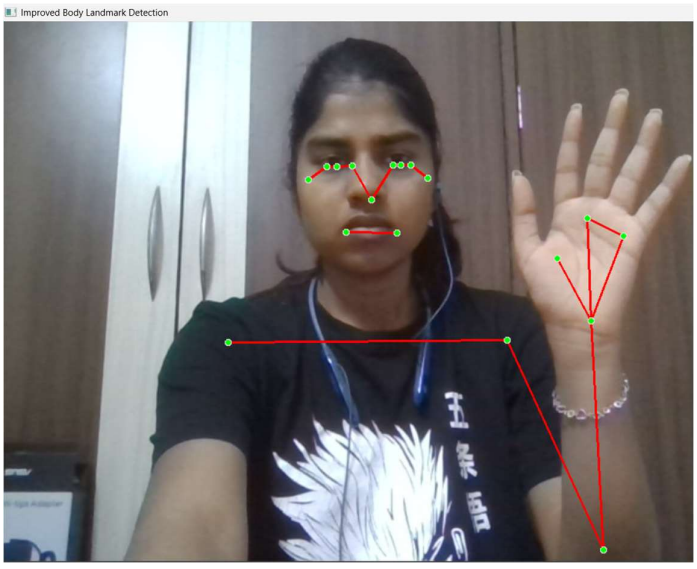
Improvement	**Purpose**
model_complexity = 2	Improved pose estimation accuracy
smooth_landmarks = True	Reduced landmark flickering
Higher detection confidence	Better landmark visibility
Higher tracking confidence	Stable movement tracking
Frame resizing	Improved performance

---

8. Performance Testing

The improved system was tested under multiple conditions.

Test Condition	Result
Full body visible	Accurate detection
Hand movement	Stable tracking
Side pose	Moderate accuracy
Fast movement	Minor landmark delay
Low lighting	Reduced visibility

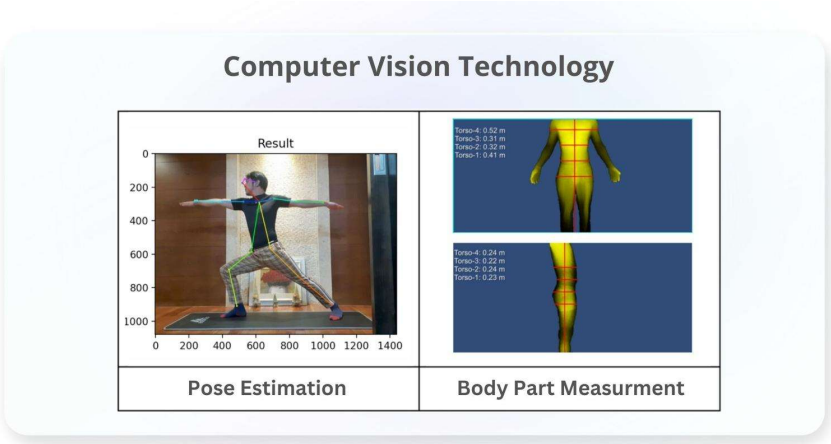


---

9. Applications of Body Landmark Detection

Applications include:

- AI fitness systems
- healthcare monitoring
- sports analysis
- rehabilitation systems
- motion tracking
- virtual try-on systems
- gesture recognition



---

10. Challenges Faced

Challenge	Solution
Landmark flickering	Enabled smooth landmark tracking
Low lighting	Improved camera positioning
Fast movement	Increased tracking confidence
Partial visibility	Optimized pose estimation settings

---

11. Learning Outcomes

During this task, the following concepts were learned:

- advanced pose estimation
- body landmark tracking
- AI pose optimization
- real-time body analysis
- skeletal visualization
- computer vision workflows
- AI tracking systems

---

12. Conclusion

The improved body landmark detection system successfully enhanced real-time pose tracking and skeletal visualization using Artificial Intelligence and Computer Vision technologies.

Compared to the Day 2 implementation, the updated system provided:

- smoother tracking
- improved landmark accuracy
- stable movement detection
- better real-time performance
- reduced landmark flickering

This task improved understanding of:

- AI pose estimation systems
- body landmark analysis
- computer vision applications
- real-time movement tracking
- AI optimization techniques